

RESULTS

Phase-3 A2A (Google Agent2Agent) — End-to-End Proof RESULTS

Run-id	phase3_a2a_20260707
Scope	ACP → A2A capability per docs / research/07.2026/00_master/ ACP_A2A_PROVIDER.md (operator decision 03_open_clarifications.md C3 — “Google A2A, NOT Zed ACP”)
Track	(T1/feature/helixllm-full-extension)
Resumed from	§11.4.147 crashed-agent respawn — the prior attempt died on a shared session-limit at “all unit tests pass, building the a2a-server binary”; preserved partial state in submodules/helix_llm/cmd/a2a-server/ + internal/a2a/ was inspected, quiescence-checked, and completed (no rewrite from scratch)
Verdict	DONE — RED → GREEN proof achieved end-to-end against the LIVE coder, no bluff
Repo (main)	e58aab9f (pre-evidence-commit HEAD)
Submodule (helix_llm)	f8632e3 (pre-code-commit HEAD)

1. §11.4.84 quiescence check (STEP 1)

Before touching anything, the preserved working tree was inspected:

- `git status --porcelain` in submodules/helix_llm showed exactly the expected untracked dirs (`cmd/a2a-server/`, `internal/a2a/`) plus **unrelated** modified/untracked entries belonging to a different, concurrent

- work stream (docs/OPERATOR_GUIDE.{md,html,pdf}, docs/qa/phase3_rag_20260707/) — these were **left untouched** (§11.4.119 single-resource-owner / §11.4.176 exclusive-claim: out of this task's scope).
- `grep` for mutation markers (MUTATED for paired, // always pass, `_mutated_*`, // MUTATION) across `cmd/a2a-server`, `internal/a2a` → **zero matches**. No mutation residue.
 - Every file in the preserved `internal/a2a` package (`types.go`, `agentcard.go`, `downstream.go`, `handlers.go`, `json.go`, `router.go`, `taskstore.go`, `a2a_test.go`) and `cmd/a2a-server/main.go` was read in full and accounted for.
 - Unit tests re-run: **all 5 PASS** (`TestAgentCardHasRequiredFields`, `TestMessageSendHappyPath`, `TestMessageSendRejectsMissingAuth`, `TestDispatchRejectsMalformedRequest`, `TestTasksGetRoundTrip`) — confirming the “unit tests pass” state the crashed agent had reached was genuinely reproducible, not assumed.
 - `go build -o bin/a2a-server ./cmd/a2a-server` → exit 0.
 - `go build ./...` (full `helix_llm` module) → exit 0. No collateral breakage from the preserved change.

One genuine gap found and completed: the preserved evidence harness at `docs/qa/phase3_a2a_20260707/harness/main.go` referenced `cmdStubServe()` in its subcommand switch (case `"stub-serve"`: `cmdStubServe()`) but the function was **never defined** — this is the exact half-finished spot the prior agent was mid-writing when it crashed (the harness would not compile). I implemented `cmdStubServe` (a deliberately-broken A2A server: empty Agent Card fields, message/send that never leaves "working" with an empty artifacts[]) so the RED-baseline step (§11.4.115) could run for real, against a live broken server over the wire, not merely as an in-process fixture. See `harness/main.go` (function `cmdStubServe`).

2. What was proven (STEP 2)

2.1 Live topology

- **Coder** (read-only, never touched): `llama-server` (`llama.cpp`), PID 2394426, `--port 18434`, model `/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf`. Confirmed reachable before AND after this session (`curl http://localhost:18434/v1/models` → HTTP 200 both times).
- **A2A server** (built + started by this session, torn down at the end): `bin/a2a-server`, listening on `:18441`, `HELIX_A2A_DOWNSTREAM_URL=http://localhost:18434` (no `/v1` suffix — the documented endpoint gotcha, `ACP_A2A_PROVIDER.md` §2.2), Bearer auth enforced via a locally-generated test token (never committed, never logged — §11.4.10).

- Ports 18434–18440 were pre-occupied by other live services and were left untouched; 18441 (A2A server) and 18442 (RED-baseline stub) were free and used exclusively by this proof.

2.2 RED baseline (§11.4.115 RED-on-broken-artifact)

A **deliberately broken** A2A server (`phase3a2a.bin stub-serve :18442`) was booted and driven with the exact SAME black-box client + analyzers used against the real server:

Probe	Result
<code>discover</code> → <code>analyze-card</code>	FAIL (<code>10_red_baseline.txt</code>) — 10 reasons: empty name/description/version/url, empty skills[], missing capabilities.* flags, empty defaultInputModes/defaultOutputModes
<code>send</code> (Fibonacci prompt) → <code>analyze-completed</code>	FAIL — task state = "working", want "completed" + artifacts[] is missing/empty

This is the RED half of the polarity pair: the analyzer genuinely rejects broken behaviour *before* being pointed at the real server.

2.3 GREEN proof against the real, live server (the definition of done)

Step	Evidence file	Result
GET <code>/.well-known/agent-card.json</code> on the real server → <code>analyze-card</code>	<code>11_green_discover.txt / .json</code>	PASS — real card: name="helixllm-co capabilities={streaming:false,p securitySchemes.bearer present
Unauthorized message/ send (no Bearer header)	<code>13_unauthorized.txt</code>	HTTP 401 , {"error":{"message":"mi analyze-rejected PASS (correctly rej
	<code>14_malformed.txt</code>	

Step	Evidence file	Result
Malformed JSON-RPC (valid Bearer, missing method)		HTTP 400 , JSON-RPC error -32600 invalid method present... → analyze-rejected PASS
Task 1 — message/ send “Write a Go function that returns the nth Fibonacci number.” (valid Bearer)	12_task1_fibonacci.json / 12_task1_send.txt	HTTP 200 , task id=9da013fb-ad0a-41... real llama-server-generated Go code implementations → analyze-completed clock.
tasks/get round-trip on task 1’s id	12b_tasksget_roundtrip.txt / 12b_task1_get.json	Returned id identical to the original, status PASS again on the round-tripped payload
Task 2 — distinct message/ send “Write a Go function named ReverseString that reverses a string.”	15_task2_reverse.json / 15_task2_send.txt	HTTP 200 , task id=b4799c8d-d714-4e... status.state="completed", real generated → analyze-completed (tokens func, n

RUNTIME SIGNATURE (matches ACP_A2A_PROVIDER.md §4 verbatim): a real A2A client performed GET / .well-known/agent-card.json → valid card with generate-code skill + Bearer security scheme; POST /a2a message/send with a real prompt + valid Bearer token; HelixLLM’s A2A server routed the Task to the **live** local coder (:18434 → POST /v1/chat/completions); the Task transitioned to state=="completed" carrying an Artifact with real, compilable-looking Go source (func + fib/reverse present, no placeholder text); the Task id round-tripped through tasks/get unchanged. **This signature is impossible to satisfy with a simulated/placeholder handler** — every step above is a captured, real over-the-wire HTTP transcript (see the .json envelope files: http_status + raw response body).

2.4 Self-validation (§11.4.107(10))

phase3a2a.bin selfvalidate (no network — pure analyzer unit check), captured in 12_self_validation.txt:

- Golden-good Agent Card → **PASS**; golden-bad (empty skills[]) → **FAIL**.
- Golden-good completed task → **PASS**; golden-bad non-terminal state → **FAIL**; golden-bad empty artifact → **FAIL**; golden-bad placeholder-artifact-missing-tokens → **FAIL**.
- Golden-good 401 rejection → **PASS**; golden-bad “wrongly processed to a completed Task” → **FAIL**.

Overall: [SELF-VALIDATION] **PASS**: analyzer **PASS**es every golden-good fixture and **FAIL**s every golden-bad fixture — the analyzer itself cannot be fooled by any of the enumerated regressed-handler shapes (§11.4.107(10) mutation-proof guarantee).

2.5 Teardown (§11.4.119 / §11.4.122)

Only our own two PIDs were stopped: bin/a2a-server (2360446) and phase3a2a.bin stub-serve (2361788) — verified by `ps -ef argv/cwd` inspection before the kill (§11.4.174 process-ownership verification). The live coder (llama-server, PID 2394426) was **never signalled** and was confirmed reachable (HTTP 200 on /v1/models) both immediately before and immediately after our teardown — see 16_teardown.txt.

3. Honest scope (§11.4.6 / §11.4.101)

Per the design’s own honest-scope note (ACP_A2A_PROVIDER.md §2.2/§4.2) and the task’s explicit instruction, this proof deliberately covers the **core task round-trip** and declares the rest out of scope rather than faking it:

- **message/stream (SSE) is NOT implemented or exercised.** The Agent Card truthfully declares `capabilities.streaming=false` — it does not advertise a capability that would fail on use.
- **Push notifications (tasks/pushNotificationConfig/*) are NOT implemented.** `capabilities.pushNotifications=false`, declared honestly. (Design open question Q4 — deferred, not silently dropped.)
- **The extended/authenticated Agent Card flow (agent/getAuthenticatedExtendedCard) is NOT implemented.** `capabilities.extendedAgentCard=false`.
- **gRPC / HTTP+JSON-REST transport bindings are NOT implemented** — only the JSON-RPC 2.0-over-HTTPS binding (spec §1.5), per the design’s stated scope.

- **File/Data Parts are accepted on decode but not exercised by this proof** — only Text Parts are sent/asserted (the `internal/a2a/types.go` doc comment states this explicitly).
- **A2A *client*** (HelixLLM/HelixAgent reaching OUT to other A2A peers, design §2.3) **is NOT implemented in this session** — only the **server** side (HelixLLM is reachable BY other A2A agents) was built + proven, which is what the resumed task explicitly asked for (Agent-Card discovery + real task round-trip against the live coder).
- **Scope-assignment note carried over from the preserved code** (documented in `internal/a2a/types.go`'s package doc, §11.4.101): the design frames the A2A server as a HelixAgent capability; this implementation instead builds it as a standalone HelixLLM binary that fronts the coder directly — the minimal, safe choice matching the actually-deployed topology (a plain-HTTP llama.cpp router container, not the full mTLS `cmd/helixllm` binary) without touching/rebuilding/restarting the live coder. This was already a documented, non-silent decision in the preserved code, carried forward unchanged.

None of the above are silently dropped: each is a truthful false capability flag or an explicit statement here, never a faked PASS.

4. Reproduce

1. Build

```
cd submodules/helix_llm
go build -o bin/a2a-server ./cmd/a2a-server
go test -v -count=1 ./internal/a2a/...
```

2. Build the black-box harness

```
cd ../../docs/qa/phase3_a2a_20260707/harness
go build -o phase3a2a.bin .
./phase3a2a.bin selfvalidate
```

3. RED baseline (broken stub on :18442)

```
./phase3a2a.bin stub-serve :18442 &
./phase3a2a.bin discover http://localhost:18442 /tmp/
    red_card.json
./phase3a2a.bin analyze-card /tmp/red_card.json    # expect FAIL,
    exit 1
kill %1
```

4. GREEN: start the real A2A server against the live coder (never restart the coder)

```
cd ../../../../submodules/helix_llm
```

```
HELIX_A2A_LISTEN_ADDR=:18441 HELIX_A2A_DOWNSTREAM_URL=http://
localhost:18434 \
HELIX_A2A_BEARER_TOKEN=$(openssl rand -hex 16) ./bin/a2a-
server &
cd ../../docs/qa/phase3_a2a_20260707/harness
./phase3a2a.bin discover http://localhost:18441 /tmp/card.json
./phase3a2a.bin analyze-card /tmp/card.json
# expect PASS, exit 0
./phase3a2a.bin send http://localhost:18441 <token> \
"Write a Go function that returns the nth Fibonacci number." /
tmp/task.json
./phase3a2a.bin analyze-completed /tmp/task.json func,fib #
expect PASS, exit 0
```

5. Sources verified (§11.4.99 / §11.4.150)

Inherited unchanged from the approved design (docs/research/07.2026/00_master/ACP_A2A_PROVIDER.md, deep-research 2026-07-07, re-confirmed against this session's implementation — no protocol fact in this document contradicts the cited sources):

- <https://a2a-protocol.org/latest/specification/>
- <https://a2a-protocol.org/v0.3.0/specification/>
- <https://github.com/a2aproject/A2A>

No new external research was required for this completion pass — the implementation follows the already-verified wire facts (Agent Card path, JSON-RPC method names, TaskState values, Part kinds) exactly as cited in the design document §1.

6. Composition footer

Composes §11.4.6 (honest scope, no-guessing) / §11.4.84 (quiescence check before touching preserved state) / §11.4.99 / §11.4.107(10) (self-validated analyzer) / §11.4.108 (four-layer: SOURCE committed → ARTIFACT builds → RUNTIME-ON-CLEAN-TARGET the §4 signature above → USER-VISIBLE a real client gets real generated code back) / §11.4.115 (RED-on-broken-artifact baseline) / §11.4.119 + §11.4.122 (never touch the live coder; own-process-only teardown) / §11.4.147 (crashed-agent respawn — resumed the preserved partial state instead of restarting from scratch) / §11.4.174 (process-ownership verified by argv/cwd before any kill).